

How to Run the Gauntlet Model

December 2025 | Liz Allyn (lizallyn@uw.edu)

This document explains how to run the Gauntlet Model tool (<https://github.com/lizallyn/TheGauntletModel>) for a new Gauntlet system, step-by-step.

Quick Start

You can run the model with pre-existing inputs to observe model outputs and mechanics using the script “Run_The_Model.R” located in the folder RunTheModel. Source the script and wait – progress updates will show which iteration the model has completed (iteration = day). A few simple plots are included to illustrate basic outputs.

To make changes to inputs and customize the model to a new location, you will need to make many adjustments to the model files, as described below.

1. Clarify Your Objectives

This modeling tool was built to help managers explore different pinniped management strategies and their *relative* impacts on salmon runs of interest. It can help illustrate the magnitude of potential benefit to salmon recovery that might be possible under different pinniped management scenarios. The tool is not intended to produce exact estimates of consumption of salmon by pinnipeds. Please feel free to reach out to Liz to make sure this tool is a good fit for your goals.

2. Define the Gauntlet

Before thinking about what the Gauntlet Model needs, you have to define what the Gauntlet is. A Gauntlet for the purposes of this model must fulfill three criteria: 1) an area where salmon migrate through, 2) there is some physical or physiological constriction that slows salmon down or limits their movement, and 3) pinnipeds have access to salmon in the Gauntlet. You should define the geographic boundaries of the Gauntlet before proceeding.

You should also consider the time scale of the model. The model is not intended to run for multiple years, so it should only be run for fewer than 365 days (a single salmon migration season).

3. Gather Data

Salmon and Pinniped Data

See the document titled “The Ingredient List.docx” for a succinct list of the data that is needed to run this model. For each population of pinnipeds, we require the abundance of the source population, which is defined as the group of pinnipeds that might enter the Gauntlet to forage at some point during the model run. For each run of salmon, we require the abundance of the run, the peak arrival date, residence time in the Gauntlet, and caloric value (can be estimated from length or weight data). Additional information about pinniped diet preferences, foraging behavior, social behavior, seasonality or movement are also helpful to verify model behavior, but not necessary to run the model.

In the Data folder, the file “salmon_run_info.csv” contains the necessary information for each salmon run.

- Peak_Date refers to the day of the year when the peak of the run arrives at the Gauntlet
- sd is the standard deviation of the arrival curve for that run of salmon
- Run_size is the total abundance of the run
- Residence is the number of days an individual from that run would be expected to spend in the Gauntlet before continuing migration
- Fish_Rate is the mortality rate due to fisheries catch
- Fishery_Open and Fishery_Close bookend the fishery (day of the year)

4. Prep Inputs

Salmon and Pinniped Info

In the folder RunTheModel, script “Prep_Salmon_Data.csv” uses the salmon run information to create an arrival curve for each salmon run. This script also defines when the fishery occurs *fishery_dates*, and the number of active pinniped harvesters each day *num_harvesters*. *Boat_days* and *fishy_days* are used by the learning model to indicate when the different cues (boats and harvest activity) are presented to pinniped predators at the Gauntlet.

The file “alpha_vals.csv” defines the search and capture efficiency of each pinniped-salmon combination. This is loaded into the model in script “Prep_Bioenergetics.R”.

The script “Prep_Bioenergetics.R” has caloric values for each pinniped species and a sampling of salmon runs. It also defines diet proportions for each predator-prey combination according to diet studies of individuals that occupy open water (non-Gauntlet) habitats. The parameter C_{max} represents the maximum consumption rate for each species of pinniped and salmon species combination, and the model allows for 2 different assumptions to be made about this value. The first assumption (low C_{max}) is that pinniped predators are consuming salmon in the Gauntlet as only a portion of their diet at a similar rate as is represented by their diets empirically measured in scat studies conducted outside of Gauntlet areas. The second C_{max} assumption (high C_{max}) represents an elevated level of consumption where pinniped predators are belly-biting, high-grading, or otherwise killing more salmon than they need to meet their daily energetic requirements. The *high_grade_factor* defines the multiplier on their energetic requirements. Parameter w represents the minimum amount of salmon that an individual must consume to define a successful foraging day. These values are currently based on the proportion of pinniped diet that was salmon (generic, not by species) measured in scats collected from individuals that were expected to have foraged in open water (non-Gauntlet areas).

In “Prep_Pinniped_Data.R”, you will need to enter the number of each species of pinniped (*num_seals* for Harbor seals, *num_zc* for California sea lions, and *num_ej* for Steller sea lions). You will also need to designate the proportion of the source population (*prop_specialists*) of harbor seals that you think could be classified as a “specialist”, or an individual that is well acquainted with the Gauntlet and likely to forage there – not naïve. The model currently assumes that all sea lions act like specialists. The seasonality of sea lion presence at the Gauntlet can be adjusted with *sealion_arrival*, which should be entered as the model iteration number, not day of year (i.e. 1 would indicate that the sea lions arrive on the first day of the model, even if the first day of the model is not January 1). Parameters for the consumption function (*gamma* and Y) are also defined here (Essington et al. 2015).

Learning Parameter Values

The model is pre-loaded with estimates for all the parameter values required by the model. It is my hope that these will continue to be updated by future research. These parameters are mostly defined in “set_pars.R”.

The Data folder contains the social association matrix for a single pinniped source population (“simulated_association_network_1.csv”) and the script that can be used to simulate more if needed (“social_associations_cleanup.R”). The original data used to simulate more networks is in “social_network_associations_zac_EMB.csv”. This simulated

matrix is cropped to represent the actual abundance of the source population of pinnipeds in the script “Prep_Pinniped_Data.R”.

Management Scenarios

The suite of management scenarios that you would like to simulate should be few and well defined. A few types of management are defined in the script “Choose_Harvest_Options.R”.

Scenario describes how the removal occur – “Boat” or “None”

Timing defines when removals occur – “Weekly” indicates a timing schedule exists, as defined by *days_gap*, and “During” indicates that removals occur during the whole period that salmon are present in the Gauntlet. Other timings can be developed and input into *harvest_schedule* as desired.

Fear tells the model whether the pinnipeds should be allowed to develop fear of harvest, depending on the method of harvest employed

Participation is the proportion of fishers participating in pinniped harvest on a given day

Catchability is parameter q

Overlap_pv is the degree of spatial overlap between harvest activity and pinniped foraging activity – what proportion of the pinnipeds in the Gauntlet are actually vulnerable to being killed during each day. *Overlap_sl* is the same for sea lions.

If you choose to employ haulout reduction management, that is also defined and implemented here via *reduction*. The function “hauloutReduction.R” reduces the abundance of pinnipeds by *reduction* and adjusts the social network to reflect those changes.

Formatting Variables

The script “initialize_variables.R” corrals all the above information into the correct formats for the model to call inputs and save outputs at each iteration.

5. Prep the Model

Many of the model dynamics were created as functions for ease of use and interpretation. The script “Run_The_Model.R” calls everything in order.

Case.study will be “Base” for all new implementations of the model until redefined.

Run_count is a leftover that will be overridden once the new model implementation is

finished, but must be 1 for the generic pre-filled model to run. *Bounds* is where you choose whether to use high Cmax or low Cmax.

Then we load all the functions required to prepare the model inputs (which are used in the input preparation described above). “makeArray.R” is used to create arrays of varying sizes to define variables and save outputs. “sampleBespoke.R” exists because the base R sample function cannot sample from a vector of length 1, which we need.

“createHarvestPlan.R” takes information about the management plans to be tested and turns them into a data frame that the model can call each day to tell it which management to implement that day. “createSalmonArrival.R” uses the salmon run info to create an arrival curve.

The model then loads the salmon and pinniped data in “Prep_Bioenergetics.R”, “Prep_Salmon_Data.R” and “Prep_Pinniped_Data.R”, and the management options described in “Choose_Harvest_Options.R”. “set_pars.R” defines parameter values, and “initialize_variables.R” creates empty output structures.

The model then loads the functions that are used to actually run the model using the script “loadLoopFunctions.R”.

6. Run the Model

The model is run in the script “The_Loop_pv.R”, though it should be run by calling this script from “Run_The_Model.R” so that the prep is completed appropriately.

Each iteration of the loop is a single model day, so the loop repeats for as many days as the model is run for (length of the salmon migration season). Here are the model components in the order they are run in the loop each day.

1. First, the model pulls the number of salmon that are still in the gauntlet that day, *daily_salmon_list*, from the running tally of gauntlet salmon abundance, *salmon_list*, and feeds that to the salmonSpeciesUpdate function to calculate the abundance of salmon in the Gauntlet after the arriving salmon are added for that day.
2. Next the model decides where each pinniped is going to forage on that day (Gauntlet or not) using their probability that has been updated at the end of the previous time step. This decision happens using the function decideForagingDestination, which does a random draw based on the probability. This first draw is just used to weed out the dead individuals.
3. The model performs social learning. First, each pinniped’s receptivity to social information is calculated for both axes of behavior using the function receptivity,

and a total receptivity is calculated. This receptivity and the pinniped's existing probability are input into the socialInfo function to calculate the P_s for that individual. The pinniped's updated probability is calculated, and their final foraging decision is decided using the function decideForagingDestination.

4. The next section calculates salmon mortality and escape using the ordinary differential equation in function run_rungeKutta. Outputs are then organized and assigned to variables. Consumed salmon (whole fish only) are assigned to individual predators that were successful at the Gauntlet (randomly).
5. Pinnipeds are removed according to the management scenario using function simpleHarvest. Pinnipeds that develop fear as a result of exposure to pinniped management are identified, and pinnipeds that are killed are tallied.
6. Pinnipeds update their probabilities according to their new experiences in updateLearning. updateLearning uses functions learnY and learnX to update learning about foraging success (X) and risk of being killed (Y). Those updated probabilities are saved in outputs so they can be called in the next iteration.

7. Visualize Results

There are a few functions that help funnel model outputs into plots in script "makePlots.R", which uses function prepForPlots. Plots of pinniped outputs are created in "Plots_Pv.R" and plots of salmon outputs are created in "Plots_salmon.R". The names of a few of these plots are included in the script "Run_The_Model.R" for ease of visualization.

Additional Info

Lots of information about parameter values and data inputs is available in the document "Parameter Descriptions and Sources.docx".

Examples of running the model for many reps in order to appropriately summarize outputs can be seen in the repo GauntletModelManipulations (<https://github.com/lizallyn/GauntletModelManipulations>).

A full description of the model, including equations for each dynamic, is available in Chapter 2 of my thesis in the Manuscripts folder, or on ResearchGate here: https://www.researchgate.net/publication/394287123_Participatory_Modeling_to_Evaluate_Tribal_Pinniped_Harvest_in_Puget_Sound_as_a_Tool_for_Salmon_Recovery